

Brio DBMS Project

Complete Study Notes — CS1211 Exam Preparation

Francis Reuben R • Darshan Jain • Harshith BA

RV University, Bengaluru • 2025–26

Contents

1	Project at a Glance	3
1.1	Three-Tier Architecture	3
1.2	What Makes This Project Non-Trivial	3
2	Database Fundamentals (Concepts You Must Know)	3
2.1	Relational Model	3
2.2	Entity-Relationship (ER) Model	3
2.3	Keys	4
2.4	Normalisation	4
3	The Brio Schema — All 13 Tables	6
3.1	The Skill-Gate Data Model	6
3.2	FK Actions You Must Know	7
4	DDL — Data Definition Language	7
4.1	Data Types Used in Brio	7
4.2	Constraints at a Glance	7
4.3	Annotated: The Three Most Important Tables	8
5	DML — Data Manipulation Language & Seed Data	9
6	The 15 Showcase Queries — Every Concept Explained	9
6.1	Q1 — GROUP BY + Aggregate Functions	9
6.2	Q2 — Multi-Table INNER JOIN	10
6.3	Q3 — Subquery with NOT IN	10
6.4	Q4 — Window Function: RANK()	10
6.5	Q5 — Common Table Expression (CTE)	10
6.6	Q6 — CASE Expression	11
6.7	Q7 — Correlated Subquery	11
6.8	Q8 — EXISTS	11
6.9	Q9 — Self-JOIN	11
6.10	Q10 — Date Functions	12

6.11	Q11 — LEFT JOIN with Computed Column	12
6.12	Q12 — HAVING with Aggregate Condition	12
6.13	Q13 — UNION	12
6.14	Q14 — DENSE_RANK()	12
6.15	Q15 — Recursive CTE	12
7	Database Views	13
7.1	View 1: learner_dashboard_view	13
7.2	View 2: project_kanban_view	14
7.3	View 3: skill_gate_compliance_view	14
8	Triggers	14
8.1	The Two-Trigger Skill-Gate Chain	15
9	Transactions and ACID	16
9.1	ACID Properties	16
9.2	Transaction Syntax	16
10	Backend Architecture	17
10.1	Express 5 Monolith	17
10.2	Dual Connection Pools	17
10.3	JWT Authentication	18
10.4	Zod Input Validation	18
10.5	All 11 API Endpoints	19
11	Frontend Architecture	19
12	Key Design Decisions — The “Why?” Questions	20
13	Complete Quick-Reference Summary	21
13.1	SQL Concepts Covered in Brio (Cheat Sheet)	21
13.2	The 15 Queries at a Glance	22
13.3	Things Most Likely to Be Asked in Viva	22

1 Project at a Glance

BRIO is a full-stack programming learning platform where learners submit code, earn AI scores, and unlock Kanban project tasks based on lesson mastery. The entire skill-gate logic — “you must pass a lesson to unlock the next task” — lives in the MySQL database as triggers, not in the application code.

1.1 Three-Tier Architecture

Tier	Technology	Role
Presentation	Vanilla JS SPA (browser)	Three tabs: Platform / Showcase / Viz
Application	Node.js + Express 5 (<code>server.js</code>)	11 REST endpoints, JWT auth, Zod validation
Data	MySQL 8 (<code>brio_db</code>)	13 tables, 3 views, 2 triggers, transactions

1.2 What Makes This Project Non-Trivial

- **Skill-gate trigger chain:** two chained triggers fire on every submission INSERT
- **Dual connection pools:** separate read-only MySQL user for the Chaos Sandbox
- **13 tables in 3NF:** strong entities, weak entities, and junction tables
- **15 live SQL queries:** window functions, CTEs, recursive CTEs served via API
- **JWT stateless auth:** token verified on every protected request

2 Database Fundamentals (Concepts You Must Know)

2.1 Relational Model

A **relation** is a table. Each **tuple** is a row. Each **attribute** is a column. A **schema** is the set of table definitions. A **database instance** is the actual data stored at a point in time.

Domain constraint: every value in a column must come from an allowed set (e.g. `ENUM('learner', 'admin')` restricts role to exactly two values).

2.2 Entity-Relationship (ER) Model

Entity: a real-world object with independent existence (e.g. `user`, `lesson`).

Weak entity: identified only in relation to another entity; its PK includes a FK to the owner (e.g. `kanban_task` needs `proj_id` to be unique).

Relationship: association between entities. **1:N** (one user owns many projects), **M:N** (many users enroll in many lessons → junction table).

Participation: *total* (every entity participates) vs *partial* (some may not).

2.2.1 Brio ER in Plain Language

- One user owns many projects (1:N, partial — not every user has a project)

- One project has many kanban_tasks (1:N, total — a task must belong to a project)
- Many users enroll in many lessons → enrollment junction table (M:N)
- One kanban_task is gated on one lesson via lesson_id FK
- One submission belongs to one user + one lesson (weak entity)

2.3 Keys

Key Type	What It Means / Brio Example
Primary Key (PK)	Uniquely identifies each row. user_id INT AUTO_INCREMENT in user.
Composite PK	PK spanning multiple columns. (user_id, lesson_id) in enrollment.
Foreign Key (FK)	References the PK of another table. owner_id in project → user.user_id.
Unique Key	Not a PK but must be unique. email and username in user.
Surrogate Key	Artificial numeric PK (AUTO_INCREMENT). Used on strong entities (sub_id).
Natural Key	Meaningful domain value. Not used in Brio's PKs but email qualifies.

Why composite PKs on junction tables? enrollment(user_id, lesson_id) — because a row in enrollment is identified by both values together. Adding a surrogate enrollment_id would allow duplicate (user, lesson) pairs, which is semantically wrong. The composite PK enforces the uniqueness constraint for free.

2.4 Normalisation

The goal: eliminate **insertion**, **deletion**, and **update anomalies** by ensuring each fact is stored in exactly one place.

2.4.1 1NF — First Normal Form

All attribute values are **atomic** (no lists, no repeating groups). Each column holds a single indivisible value.

Violation example: storing tags = "python,oop,functions" in lesson.

Brio fix: separate lesson_tag(lesson_id, tag) table — each tag is its own row.

2.4.2 2NF — Second Normal Form

1NF + every non-key attribute is **fully functionally dependent** on the *entire* primary key (no partial dependency).

Only applies to composite PKs.

Example: if enrollment had a column lesson_title, that depends only on lesson_id (partial dependency) → move it to the lesson table.

Brio: enrollment(user_id, lesson_id, enrolled_at) — enrolled_at depends on the full composite PK, so it's in 2NF.

2.4.3 3NF — Third Normal Form

2NF + no **transitive dependency** (non-key attribute depends on another non-key attribute).

Example: if lesson stored subject_name alongside subject_id, then lesson_id → subject_id → subject_name is a transitive dependency.

Brio fix: subject_name lives in the subject table; lesson only stores subject_id FK. Query it with a JOIN when needed.

All 13 Brio tables are in 3NF. The payoff: when a subject name changes, you update one row in subject and every lesson that references it automatically reflects the change.

3 The Brio Schema — All 13 Tables

Table	Type	Purpose & Key Points
user	Strong	Central entity. PK: user_id. role ENUM('learner', 'admin'). Passwords bcrypt-hashed.
lesson	Strong	Lesson catalogue. FK → subject, programming_language. difficulty ENUM.
subject	Strong	Subject master (Python, Web Dev...). subject_name UNIQUE.
programming_language	Strong	Language master (Python, JS...). Referenced by lesson.
project	Dependent	Owned by one user (FK owner_id). status ENUM('active', 'shipped', 'archived').
kanban_task	Weak	Linked to project + gated on lesson. status ENUM('locked', 'in_progress', 'done').
submission	Weak	Event source for triggers. Stores ai_score TINYINT(0-100).
lesson_tag	Multivalued	Tags per lesson (1NF compliance). PK: (lesson_id, tag).
project_milestone	Dependent	Milestones with optional depends_on self-FK (used in Q15 recursive CTE).
enrollment	Junction	M:N user ↔ lesson. PK: (user_id, lesson_id).
team_member	Junction	M:N user ↔ project with role attribute.
lesson_prerequisite	Junction	M:N lesson ↔ lesson (prerequisite chain). Used in Q9 self-JOIN.
user_badge	Junction	Badges earned by users. PK: (user_id, badge_name).

3.1 The Skill-Gate Data Model

The key relationship that powers everything:

`kanban_task.lesson_id → lesson.lesson_id`

This FK says: “*this Kanban task is gated behind this lesson.*” When a user submits a lesson with `ai_score ≥ 70`, the trigger looks for all `kanban_task` rows where `lesson_id` matches AND `proj_id` belongs to the submitting user’s projects, then flips status from ‘locked’ to ‘in_progress’.

3.2 FK Actions You Must Know

Policy	Behaviour / Where Used in Brio
CASCADE	Delete/update parent → delete/update all children. project(owner_id) cascades on user delete.
RESTRICT	Prevent parent delete if children exist. lesson(subject_id) — can’t delete a subject that has lessons.
SET NULL	Set FK to NULL on parent delete. kanban_task(lesson_id) — task survives if lesson is deleted (just loses its gate).

4 DDL — Data Definition Language

4.1 Data Types Used in Brio

- `INT AUTO_INCREMENT` — surrogate PKs
- `VARCHAR(n)` — strings with max length
- `TEXT` — unbounded text (bio, content)
- `TINYINT UNSIGNED` — 0–255, used for `ai_score` (0–100)
- `ENUM('a', 'b')` — type-safe categorical values
- `TIMESTAMP DEFAULT CURRENT_TIMESTAMP` — auto-records creation time
- `DATE` — date only, no time (deadline)

4.2 Constraints at a Glance

Constraint	What It Does
PRIMARY KEY	Unique + NOT NULL identifier. Can be single or composite.
NOT NULL	Column must always have a value.
UNIQUE	Value must be unique across all rows (email, username).
FOREIGN KEY	References PK of another table; enforces referential integrity.
ENUM	Column value must be one of a fixed set. Database rejects invalid values.
DEFAULT	Value used if none supplied (<code>role DEFAULT 'learner'</code>).
INDEX	Speeds up WHERE/JOIN queries on that column. Not a constraint per se.

4.3 Annotated: The Three Most Important Tables

Listing 1: user table — central auth entity

```
CREATE TABLE user (
  user_id      INT AUTO_INCREMENT PRIMARY KEY,  -- surrogate PK
  username     VARCHAR(50)  NOT NULL UNIQUE,    -- login handle
  email        VARCHAR(100) NOT NULL UNIQUE,    -- contact + alt login
  password     VARCHAR(255) NOT NULL,           -- NEVER plaintext;
              bcrypt hash
  role         ENUM('learner','admin') DEFAULT 'learner',
  bio          TEXT,                             -- nullable
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_user_email (email)                  -- speeds up login
              lookup
) ENGINE=InnoDB;                               -- InnoDB = FK support +
              transactions
```

Listing 2: kanban_task — the skill-gate pivot

```
CREATE TABLE kanban_task (
  task_id      INT AUTO_INCREMENT PRIMARY KEY,
  proj_id      INT NOT NULL,
  lesson_id    INT,                             -- NULL = no gate (free task)
  title        VARCHAR(200) NOT NULL,
  status       ENUM('locked','in_progress','done') DEFAULT 'locked',
  priority     INT DEFAULT 5,
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (proj_id) REFERENCES project(proj_id) ON DELETE
              CASCADE,
  FOREIGN KEY (lesson_id) REFERENCES lesson(lesson_id) ON DELETE SET
              NULL
  -- SET NULL so deleting a lesson doesn't cascade-delete all tasks
) ENGINE=InnoDB;
```

Listing 3: submission — trigger event source

```
CREATE TABLE submission (
  sub_id       INT AUTO_INCREMENT PRIMARY KEY,
  user_id      INT NOT NULL,
  lesson_id    INT NOT NULL,
  ai_score     TINYINT UNSIGNED NOT NULL,      -- CHECK: 0-100 via
              UNSIGNED + app validation
  submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES user(user_id) ON DELETE
              CASCADE,
```



```
FOREIGN KEY (lesson_id) REFERENCES lesson(lesson_id) ON DELETE
    RESTRICT
-- RESTRICT: don't delete a lesson that has submission records
) ENGINE=InnoDB;
```

Why InnoDB? The MyISAM engine doesn't support foreign keys or transactions. InnoDB (MySQL's default) supports both, making it the only valid choice for a schema with 13 FK relationships and ACID-compliant transactions.

5 DML — Data Manipulation Language & Seed Data

Seed data strategy: insert in dependency order — strong entities first, then dependent/weak entities, then junction tables. You cannot insert a `kanban_task` before the project it belongs to exists.

Insertion order: `subject` → `programming_language` → `user` → `lesson` → `project` → `kanban_task` → `enrollment` → `submission` → `team_member`

bcrypt password hashing: passwords are never stored as plaintext. `bcrypt.hash(password, 10)` produces a 60-character hash like `$2b$10$...`. During login, `bcrypt.compare(input, hash)` returns `true/false`. The salt rounds (10) control how slow the hash is — making brute-force expensive.

6 The 15 Showcase Queries — Every Concept Explained

Each query teaches one or more SQL concepts. Know *what* the concept is, *why* it's used, and *what* the query returns.

6.1 Q1 — GROUP BY + Aggregate Functions

`GROUP BY` collapses all rows with the same value into one summary row. Aggregate functions (`COUNT`, `AVG`, `MAX`, `MIN`, `SUM`) operate over each group. `LEFT JOIN` ensures lessons with *zero* submissions still appear (result: `total_submissions = 0`).

```
SELECT l.lesson_id, l.title,
       COUNT(s.sub_id)           AS total_submissions,
       ROUND(AVG(s.ai_score), 2) AS average_score,
       MAX(s.ai_score)           AS highest_score
FROM lesson l
LEFT JOIN submission s ON l.lesson_id = s.lesson_id
GROUP BY l.lesson_id, l.title
ORDER BY total_submissions DESC;
```

INNER vs LEFT JOIN: `INNER JOIN` would omit lessons with no submissions. `LEFT JOIN` keeps all lessons and puts `NULL` for aggregates when no submissions exist. Always ask: “do I want rows

with no matching records?” → YES → LEFT JOIN.

6.2 Q2 — Multi-Table INNER JOIN

INNER JOIN returns only rows that have a match in BOTH tables. Join multiple tables by chaining JOIN ... ON ... clauses. Alias each table to keep columns readable.

Purpose: learner profiles with their enrolment counts — join user, enrollment, and lesson in one query.

6.3 Q3 — Subquery with NOT IN

A *subquery* is a SELECT inside another SELECT. NOT IN (SELECT ...) filters rows that do NOT appear in the inner result set.

```
SELECT lesson_id, title FROM lesson
WHERE lesson_id NOT IN (
    SELECT DISTINCT lesson_id FROM submission
);
-- Returns lessons that NO learner has ever submitted
```

Pitfall: If the subquery returns any NULL, NOT IN behaves unexpectedly (everything fails). Use NOT EXISTS when NULLs are possible. Here lesson_id is a FK — it can't be NULL — so NOT IN is safe.

6.4 Q4 — Window Function: RANK()

A *window function* computes a value across a “window” of rows related to the current row, without collapsing rows (unlike GROUP BY).

RANK() OVER (ORDER BY col DESC) assigns rank 1 to the highest, 2 to next, etc. Ties get the same rank; the next rank skips (1, 1, 3...).

PARTITION BY restarts ranking per group.

```
SELECT u.username,
       ROUND(AVG(s.ai_score), 2) AS avg_score,
       RANK() OVER (ORDER BY AVG(s.ai_score) DESC) AS performance_rank
FROM user u
JOIN submission s ON u.user_id = s.user_id
WHERE u.role = 'learner'
GROUP BY u.user_id, u.username
ORDER BY performance_rank;
```

Window functions (RANK, DENSE_RANK, ROW_NUMBER, LAG, LEAD) require the OVER clause. They were introduced in SQL:2003 and are available in MySQL 8+. This is one reason the project specifies MySQL 8.

6.5 Q5 — Common Table Expression (CTE)

A **CTE** (`WITH name AS (...)`) defines a named temporary result set valid for one query. It improves readability and allows referencing the same subquery multiple times without repeating it.

```
WITH high_performers AS (
    SELECT u.user_id, u.username, u.email,
           ROUND(AVG(s.ai_score), 2) AS avg_score
    FROM user u JOIN submission s ON u.user_id = s.user_id
    GROUP BY u.user_id, u.username, u.email
    HAVING AVG(s.ai_score) >= 85           -- filter: only >=85 avg
)
SELECT hp.*, COUNT(e.lesson_id) AS enrolled_lessons
FROM high_performers hp
LEFT JOIN enrollment e ON hp.user_id = e.user_id
GROUP BY hp.user_id, hp.username, hp.email, hp.avg_score
ORDER BY hp.avg_score DESC;
```

6.6 Q6 — CASE Expression

`CASE WHEN ... THEN ... ELSE ... END` is SQL's if-else. It creates a computed column from conditional logic. Used here to bucket raw scores into labelled performance bands.

```
CASE
    WHEN s.ai_score >= 90 THEN 'Excellent'
    WHEN s.ai_score >= 70 THEN 'Good'
    WHEN s.ai_score >= 50 THEN 'Average'
    ELSE 'Needs Work'
END AS performance_band
```

6.7 Q7 — Correlated Subquery

A *correlated subquery* references a column from the outer query — it is re-executed once per outer row (unlike a regular subquery which runs once). Used here to find each learner's *latest* submission per lesson.

```
WHERE s.submitted_at = (
    SELECT MAX(s2.submitted_at) FROM submission s2
    WHERE s2.user_id = s.user_id AND s2.lesson_id = s.lesson_id
)
-- s2 references s from the outer query -- that's correlation
```

6.8 Q8 — EXISTS

`EXISTS (subquery)` returns TRUE if the subquery returns at least one row. Unlike `IN`, it short-circuits on the first match (faster for large sets). Used here: “find learners enrolled in at least one project.”

6.9 Q9 — Self-JOIN

A **self-JOIN** joins a table to itself by aliasing it twice. Used here on `lesson_prerequisite` to display prerequisite chains: “Lesson B requires Lesson A” → join lesson to itself via the prerequisite table.

6.10 Q10 — Date Functions

```
WHERE s.submitted_at >= NOW() - INTERVAL 7 DAY
-- Returns submissions from the last 7 days
-- NOW() = current datetime; DATE_SUB(NOW(), INTERVAL 7 DAY) is
   equivalent
```

6.11 Q11 — LEFT JOIN with Computed Column

Joins project to kanban_task and computes completion percentage:

```
ROUND(SUM(kt.status = 'done') * 100.0 / NULLIF(COUNT(kt.task_id), 0),
      1)
AS completion_pct
-- SUM(condition) counts TRUEs (MySQL treats TRUE=1, FALSE=0)
-- NULLIF prevents division by zero (returns NULL instead)
```

6.12 Q12 — HAVING with Aggregate Condition

WHERE filters rows *before* grouping. HAVING filters *after* grouping, so it can use aggregate functions. “Find subjects where the average score across all submissions is below 60.”

```
GROUP BY l.subject_id, s.subject_name
HAVING AVG(sub.ai_score) < 60
```

6.13 Q13 — UNION

UNION stacks two result sets vertically. Both must have the same number of columns with compatible types. UNION removes duplicates; UNION ALL keeps them. Used here: combine learner and admin rosters into a single unified list.

6.14 Q14 — DENSE_RANK()

DENSE_RANK() is like RANK() but with *no gaps*: ties get the same rank and the next rank is consecutive (1, 1, 2... vs RANK’s 1, 1, 3...). Used here: top 3 lessons by submission volume.

6.15 Q15 — Recursive CTE

A *recursive CTE* calls itself. Structure:

- **Anchor member:** the base case (starting rows)
- **Recursive member:** joins the CTE to itself to go one level deeper
- Terminates when no new rows are added

Used here: traverse the `project_milestone` dependency chain — milestone B depends on milestone A, which depends on milestone Root.

```

WITH RECURSIVE milestone_chain AS (
  -- Anchor: root milestones (no dependencies)
  SELECT milestone_id, title, depends_on, 0 AS depth
  FROM project_milestone
  WHERE depends_on IS NULL
  UNION ALL
  -- Recursive: each milestone that depends on one already in the CTE
  SELECT pm.milestone_id, pm.title, pm.depends_on, mc.depth + 1
  FROM project_milestone pm
  JOIN milestone_chain mc ON pm.depends_on = mc.milestone_id
)
SELECT * FROM milestone_chain ORDER BY depth, milestone_id;

```

7 Database Views

A **view** is a named, stored SELECT query. When you query a view, MySQL executes the underlying SELECT dynamically — views always reflect current data. Benefits: encapsulate complex JOINS, simplify API queries, restrict column visibility.

Why use views in Brio? Each API endpoint executes one simple `SELECT * FROM view WHERE user_id = ?`. Without views, every endpoint would repeat complex 4-table JOINS. Views also make the backend code readable and testable independently of the SQL.

7.1 View 1: learner_dashboard_view

Used by GET `/api/profile/me`. Aggregates per-learner stats.

```

CREATE OR REPLACE VIEW learner_dashboard_view AS
SELECT u.user_id, u.username, u.email, u.bio,
       COUNT(DISTINCT e.lesson_id) AS enrolled_courses, -- DISTINCT
       avoids dup count
       ROUND(AVG(s.ai_score), 2) AS avg_progress,
       COUNT(s.sub_id) AS total_submissions,
       ROUND(AVG(CASE WHEN s.ai_score IS NOT NULL
                     THEN s.ai_score END), 2) AS avg_score
FROM user u
LEFT JOIN enrollment e ON u.user_id = e.user_id -- LEFT: include
users with 0 enrollments
LEFT JOIN submission s ON u.user_id = s.user_id -- LEFT: include
users with 0 submissions
WHERE u.role = 'learner'
GROUP BY u.user_id, u.username, u.email, u.bio;

```

7.2 View 2: project_kanban_view

Used by GET /api/projects. Per-project task breakdown.

```
CREATE OR REPLACE VIEW project_kanban_view AS
SELECT p.proj_id, p.title, p.status, p.deadline,
       COUNT(kt.task_id)                AS total_tasks,
       SUM(kt.status = 'done')          AS done_tasks,
       SUM(kt.status = 'in_progress')  AS in_progress_tasks,
       SUM(kt.status = 'locked')        AS locked_tasks,
       ROUND(SUM(kt.status = 'done') * 100.0
             / NULLIF(COUNT(kt.task_id), 0), 1) AS completion_pct
FROM project p
LEFT JOIN kanban_task kt ON p.proj_id = kt.proj_id
GROUP BY p.proj_id, p.title, p.status, p.deadline;
-- completion_pct is DERIVED at query time -- never stored, never stale
```

7.3 View 3: skill_gate_compliance_view

Used by the Showcase module. Shows *why* each locked task is locked.

```
CREATE OR REPLACE VIEW skill_gate_compliance_view AS
SELECT kt.task_id, kt.title AS task_title, kt.status AS task_status,
       l.title AS gate_lesson, u.username,
       MAX(s.ai_score) AS best_score,
       CASE
         WHEN MAX(s.ai_score) >= 70 THEN 'Eligible to unlock'
         WHEN MAX(s.ai_score) IS NULL THEN 'Lesson not submitted yet'
         ELSE 'Score too low (< 70 required)'
       END AS gate_status
FROM kanban_task kt
JOIN project p ON kt.proj_id = p.proj_id
JOIN user u ON p.owner_id = u.user_id
JOIN lesson l ON kt.lesson_id = l.lesson_id
LEFT JOIN submission s ON s.lesson_id = kt.lesson_id AND s.user_id = u.
    user_id
WHERE kt.status = 'locked'
GROUP BY kt.task_id, kt.title, kt.status, l.title, u.username;
```

8 Triggers

A **trigger** is a stored procedure that fires automatically when a specified DML event (INSERT, UPDATE, DELETE) occurs on a table.

Syntax: CREATE TRIGGER name {BEFORE|AFTER} {INSERT|UPDATE|DELETE} ON table FOR EACH ROW BEGIN ... END

NEW.column = the new row value (INSERT/UPDATE).

OLD.column = the old row value (UPDATE/DELETE).

8.1 The Two-Trigger Skill-Gate Chain

The complete flow when a learner submits a lesson:

1. API receives POST /api/submit with {lesson_id, ai_score}
2. INSERT INTO submission (user_id, lesson_id, ai_score)
3. **Trigger 1 fires** (AFTER INSERT on submission)
4. If NEW.ai_score >= 70: UPDATE all kanban_task rows WHERE status='locked' AND lesson_id=NEW.lesson_id AND task belongs to submitter's projects → set status='in_progress'
5. **Trigger 2 fires** (AFTER UPDATE on kanban_task)
6. If ALL tasks in the project are status='done': UPDATE project SET status='shipped'

Listing 4: Trigger 1: unlock tasks on qualifying score

```
DELIMITER $$
CREATE TRIGGER trg_unlock_task_after_submission
AFTER INSERT ON submission
FOR EACH ROW
BEGIN
    IF NEW.ai_score >= 70 THEN
        UPDATE kanban_task
        SET     status = 'in_progress'
        WHERE  status   = 'locked'
              AND lesson_id = NEW.lesson_id      -- matches the lesson
              just submitted
              AND proj_id IN (
                  SELECT proj_id FROM project
                  WHERE  owner_id = NEW.user_id  -- only THIS user's
                  projects
              );
    END IF;
END$$
DELIMITER ;
```

Listing 5: Trigger 2: auto-ship project when all tasks done

```
DELIMITER $$
CREATE TRIGGER trg_auto_ship_project
AFTER UPDATE ON kanban_task
FOR EACH ROW
BEGIN
    DECLARE remaining_tasks INT;
    SELECT COUNT(*) INTO remaining_tasks
    FROM    kanban_task
```

```

WHERE proj_id = NEW.proj_id AND status != 'done';

IF remaining_tasks = 0 THEN
    UPDATE project SET status = 'shipped'
    WHERE proj_id = NEW.proj_id;
END IF;
END$$
DELIMITER ;

```

Why enforce the skill-gate in the database (trigger) rather than in the Node.js API code?

- Application code can be bypassed (direct DB insert, script, admin tool)
- A trigger fires on *every* INSERT regardless of what caused it
- The rule is a *data integrity rule* — it belongs in the data layer
- Separation of concerns: the business rule isn't scattered across endpoints

MySQL limitation: a trigger cannot modify the same table it fires on (prevents infinite loops). Trigger 1 modifies `kanban_task`; Trigger 2 fires on `kanban_task` and modifies `project` — no circular reference.

9 Transactions and ACID

9.1 ACID Properties

Property	Meaning / Brio Example
Atomicity	Either ALL operations in a transaction commit or NONE do. Enrollment + submission must both succeed or both fail.
Consistency	DB moves from one valid state to another. FK constraints remain satisfied after commit.
Isolation	Concurrent transactions don't see each other's intermediate states. Two users submitting simultaneously don't interfere.
Durability	Once committed, changes survive crashes (InnoDB writes to disk via redo log).

9.2 Transaction Syntax

```

START TRANSACTION;
SAVEPOINT before_enrollment;           -- snapshot point

INSERT INTO enrollment (user_id, lesson_id, enrolled_at)
VALUES (1, 3, NOW());

```



```
-- Check for duplicate
IF (SELECT COUNT(*) FROM enrollment WHERE user_id=1 AND lesson_id=3) >
  1 THEN
  ROLLBACK TO SAVEPOINT before_enrollment;    -- undo only to
  savepoint
  SELECT 'Error: duplicate enrollment' AS result;
ELSE
  RELEASE SAVEPOINT before_enrollment;        -- discard savepoint
  COMMIT;                                     -- persist changes
  SELECT 'Enrolled successfully' AS result;
END IF;
```

SAVEPOINT allows partial rollback. `ROLLBACK TO SAVEPOINT name` undoes everything after the savepoint but keeps earlier operations. `ROLLBACK` (no savepoint) undoes the entire transaction.

10 Backend Architecture

10.1 Express 5 Monolith

Everything lives in `website/server.js`. One file = all 11 route handlers + middleware + pool setup. This is intentional for educational clarity. In production you'd split into `routes/`, `controllers/`, `services/`, `repositories/`.

10.2 Dual Connection Pools

Pool	MySQL User	Used By
Primary (pool)	read-write user	All 9 protected API endpoints
Read-only (readOnlyPool)	SELECT-only user	/api/showcase/chaos-sandbox only

```
const pool = mysql.createPool({ // 10 connections, read-write
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  connectionLimit: 10,
  // ...
});

const readOnlyPool = mysql.createPool({ // 5 connections, SELECT-only
  user: process.env.DB_READONLY_USER, // MySQL user has no INSERT/
    UPDATE/DELETE grants
  connectionLimit: 5,
});
```

Why a separate read-only pool for the Chaos Sandbox? The Chaos Sandbox lets anyone type arbitrary SQL. Two defence layers:

1. Server-side: reject any query not starting with SELECT (easily bypassed)
2. Database-level: the MySQL user itself has no INSERT/UPDATE/DELETE grants

Layer 2 is the real guarantee. Even if a crafty user bypasses the Node.js check via a subquery trick, the DB refuses to execute DML.

10.3 JWT Authentication

JSON Web Token (JWT): a stateless authentication token. Structure: Header.Payload.Signature (base64-encoded, dot-separated).

- **Stateless:** the server doesn't store sessions — the token carries the user info
- **Expiry:** Brio tokens expire after 24 hours
- **Signature:** signed with JWT_SECRET — tampered tokens fail verification

```
// Login: verify password -> issue token
const token = jwt.sign(
  { userId: user.user_id, role: user.role },
  process.env.JWT_SECRET,
  { expiresIn: '24h' }
);

// Middleware: verify token on every protected route
function authenticate(req, res, next) {
  const token = req.headers['authorization']?.slice(7); // strip "
    Bearer "
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = { id: decoded.userId, role: decoded.role };
    next();
  } catch {
    res.status(403).json({ error: 'Invalid or expired token' });
  }
}
```

10.4 Zod Input Validation

Zod validates request bodies at runtime with a schema. If the body doesn't match, Zod throws a `ZodError` which the global error handler converts to 400 Validation failed {details}. This prevents malformed data reaching the database layer.

```
const { ai_score, lesson_id } = z.object({
  ai_score: z.number().int().min(0).max(100),
  lesson_id: z.number().int().positive(),
}).parse(req.body);
```

```
// If parse() throws, the global error handler returns 400
  automatically
```

10.5 All 11 API Endpoints

Route	Auth	Purpose
POST /api/auth/login	None	bcrypt verify → JWT
GET /api/profile/me	JWT	learner_dashboard_view
POST /api/profile/update	JWT	Update bio
GET /api/projects	JWT	project_kanban_view
GET /api/kanban/:projectId	JWT	Tasks by priority
GET /api/lessons	JWT	Enrolled lessons + scores
POST /api/submit	JWT	INSERT submission → triggers fire
GET /api/showcase/stats	None	Live user/project/submission counts
GET /api/showcase/queries	None	Descriptor list for 15 queries
POST /api/showcase/query	None	Execute query by ID (Q1–Q15, V1–V3)
POST /api/showcase/chaos-sandbox	None	Arbitrary SELECT via read-only pool

11 Frontend Architecture

Vanilla JS SPA: no React, no Vue, no build step. All JS runs directly in the browser from `website/public/`. Three “tabs” are really three `<div>` sections shown/hidden based on `window.location.hash`.

Tab	Auth?	What it shows
Platform	JWT required	Learner dashboard: stats, kanban board, lesson list
Showcase	None	Live query runner (Q1-Q15), view demos, Chaos Sandbox
System Viz	None	Static architecture diagrams

Client-side state in localStorage:

- `jwt_token` — JWT string (cleared on logout)
- `user_profile` — cached profile data
- `theme` — dark/light preference

Hash router: `window.addEventListener('hashchange', renderTab)` — no server-side routing

needed; all navigation is client-side.

12 Key Design Decisions — The “Why?” Questions

These are the questions most likely to come up in a viva. Have a crisp one-sentence answer and a follow-up explanation ready.

Q: Why is the skill-gate enforced in a trigger and not in the API code?

A: Application code can be bypassed; triggers cannot. Any INSERT to `submission` — regardless of source — fires the trigger unconditionally. The pedagogical rule ($\text{score} \geq 70$ unlocks the task) is a data integrity rule and belongs in the data layer.

Q: Why use views instead of writing the JOINS directly in the API endpoints?

A: Views encapsulate complex logic in one place. If the schema changes, you update the view definition, not every endpoint that uses it. They also act as an abstraction layer — the API just does `SELECT * FROM view WHERE id = ?`.

Q: Why are composite PKs used on junction tables instead of surrogate keys?

A: A surrogate `enrollment_id` would allow duplicate (user, lesson) pairs — e.g. the same user enrolling in the same lesson twice — which is semantically wrong. The composite PK (`user_id`, `lesson_id`) enforces uniqueness for free, directly reflecting the relational model.

Q: What is the Chaos Sandbox and why is it safe?

A: It lets anyone type arbitrary SQL SELECT queries. It’s safe via two layers: (1) server-side check rejects any query not starting with SELECT; (2) the MySQL user for the read-only pool has no DML privileges — even if you bypass the Node.js check, the database refuses to execute anything other than SELECT.

Q: Why InnoDB and not MyISAM?

A: InnoDB supports foreign keys and ACID transactions. MyISAM doesn’t. Since Brio has 13 FK relationships and uses transactions for enrollment, InnoDB is the only viable choice.

Q: Explain the normalisation of the lesson table.

A: Storing `subject_name` directly in `lesson` would be a transitive dependency (`lesson_id` → `subject_id` → `subject_name`) — that’s a 3NF violation. Brio stores only `subject_id` FK in `lesson` and retrieves the name via JOIN. If the subject name changes, one row in `subject` is updated; all lessons reflect it automatically.

Q: Why store `lesson_tag` in a separate table instead of a VARCHAR column?

A: Storing tags = "python,oop,closures" in one column violates 1NF (non-atomic values). You can’t query “all lessons tagged python” without string parsing. The `lesson_tag(lesson_id, tag)` table gives each tag its own row — fully atomic, indexable, and queryable.

Q: What is JWT and why is it stateless?

A: JWT is a self-contained token with a payload (user ID, role) and a cryptographic signature. The server verifies the signature using the shared JWT_SECRET — it doesn't need to query a sessions table. This scales well (no session state to distribute) and the 24-hour expiry limits exposure.

Q: What does ACID mean? Give an example from Brio.

A: Atomicity (enrollment + submission both commit or neither does), Consistency (FK constraints stay satisfied), Isolation (concurrent submissions don't interfere), Durability (committed data survives a crash via InnoDB's redo log). The enrollment transaction with savepoint-based rollback demonstrates atomicity and consistency.

Q: What does LEFT JOIN do that INNER JOIN doesn't?

A: LEFT JOIN returns all rows from the left table, with NULL for columns from the right table when there's no match. INNER JOIN only returns rows that match in both. In Brio, learner_dashboard_view uses LEFT JOINs so learners with zero enrollments or zero submissions still appear in the view (with NULLs for their stats).

Q: Difference between RANK() and DENSE_RANK()?

A: If two rows tie for rank 2, RANK() assigns both rank 2 and skips to rank 4 (gap). DENSE_RANK() assigns both rank 2 and continues with rank 3 (no gap). Brio uses RANK() in Q4 (learner performance) and DENSE_RANK() in Q14 (top 3 lessons).

Q: What is a correlated subquery? How is it different from a regular subquery?

A: A regular subquery is evaluated once and its result is reused. A correlated subquery references a column from the outer query — it is re-evaluated once *per outer row*. Q7 in Brio uses a correlated subquery to find the latest submission timestamp per learner per lesson, where the inner query references s.user_id and s.lesson_id from the outer query.

13 Complete Quick-Reference Summary

13.1 SQL Concepts Covered in Brio (Cheat Sheet)

- **DDL:** CREATE TABLE, constraints, ENUM, INDEX
- **DML:** INSERT (seed data, bcrypt)
- **SELECT:** WHERE, ORDER BY, LIMIT
- **JOINS:** INNER, LEFT, SELF, 4-table
- **Aggregates:** COUNT, AVG, MAX, SUM, GROUP BY
- **HAVING:** filter after grouping
- **Subqueries:** scalar, IN, NOT IN, EXISTS
- **Correlated subquery:** re-evaluated per outer row
- **CASE:** conditional computed column
- **Date functions:** NOW(), INTERVAL, DATE_SUB()
- **UNION:** stack two result sets
- **Window functions:** RANK(), DENSE_RANK()

- OVER
- **CTEs:** WITH clause, non-recursive
- **Recursive CTEs:** WITH RECURSIVE
- **Views:** CREATE OR REPLACE VIEW
- **Triggers:** AFTER INSERT, AFTER UPDATE, NEW/OLD
- **Transactions:** START/COMMIT/ROLLBACK/SAVEPOINT
- **NULLIF:** prevent division-by-zero
- **ROUND():** decimal rounding
- **FK policies:** CASCADE, RESTRICT, SET NULL

13.2 The 15 Queries at a Glance

Q	Concept	One-line Description
Q1	GROUP BY + AVG/COUNT	Submissions per lesson with avg score
Q2	Multi-JOIN	Learner profiles with enrolment count
Q3	NOT IN subquery	Lessons never submitted by anyone
Q4	RANK() OVER	Learner leaderboard by avg score
Q5	CTE (WITH)	High-performers ($\text{avg} \geq 85$)
Q6	CASE expression	Score banding (Excellent/Good/Average/Needs Work)
Q7	Correlated subquery	Latest submission per learner per lesson
Q8	EXISTS	Learners in at least one project
Q9	Self-JOIN	Lesson prerequisite chains
Q10	Date functions	Submissions in last 7 days
Q11	LEFT JOIN + NULLIF	Projects with task completion %
Q12	HAVING	Subjects with avg score below 60
Q13	UNION	Combined learner + admin roster
Q14	DENSE_RANK()	Top 3 lessons by submission volume
Q15	Recursive CTE	Project milestone dependency traversal

13.3 Things Most Likely to Be Asked in Viva

1. *Walk me through what happens when a learner submits a lesson.*
 → POST /api/submit → Zod validates → INSERT submission → Trigger 1 fires → if score ≥ 70 , tasks unlocked → Trigger 2 fires → if all tasks done, project ships.
2. *What is the skill-gate mechanism?*
 → kanban_task has a lesson_id FK. Trigger 1 checks this FK to decide which tasks to unlock. The database enforces the rule, not the app code.
3. *Why does your project need two connection pools?*
 → Security. The Chaos Sandbox gets a read-only MySQL user. Even if the Node.js check is bypassed, the DB user can only SELECT.
4. *What is 3NF? Give an example from your project.*
 → No transitive dependencies. subject_name is in the subject table, not lesson. lesson stores only

subject_id FK. Avoids update anomalies.

5. *Explain the difference between a view and a table.*

→ A table stores data. A view stores a query definition — its data is derived at query time. Views are always current; you can't directly INSERT into most views.

6. *What is ACID? Is it guaranteed in Brio?*

→ Yes, because InnoDB engine is used and transactions wrap multi-step operations. The enrollment transaction demonstrates all four properties.

7. *What's the difference between WHERE and HAVING?*

→ WHERE filters rows before grouping; HAVING filters after. HAVING can use aggregate functions; WHERE cannot.

8. *Why do you use LEFT JOIN in the dashboard view?*

→ To include learners who haven't enrolled in any lessons or haven't submitted anything yet. INNER JOIN would hide them.

9. *Explain the recursive CTE in Q15.*

→ The anchor selects root milestones (depends_on IS NULL). The recursive part joins project_milestone to the CTE itself to find milestones that depend on already-found milestones. Terminates when no new rows match.

10. *How does bcrypt work?*

→ bcrypt(password, saltRounds) generates a one-way hash. On login, bcrypt.compare(input, storedHash) re-hashes the input with the embedded salt and compares. Even if the DB is leaked, plaintext can't be recovered.

Golden Rule for the Viva: for every design decision, always say *why*, not just *what*. “We used a trigger *because* it fires regardless of which application path inserts the row” is the answer; “we used a trigger” is not.